PyTorch Community Nepal

Accelerating LLM Inference with Speculative Decoding at the Edge

1st November 2026

Manjil Nepal

Research Associate, Indian Institute of Science, Bangalore

manjil.nepal03@gmail.com

manjilnepal.github.io

Previously

Research Intern, Agency for Science, Technology and Research, Singapore

Research Intern, Indian Institute of Technology Dhanbad, India

Contents

1 LLM Basics and Popular LLMs

2 LLM Inference and Popular Inference Engines

3 Understanding Edge Devices

4 Multi-Tenant Edge–Server Architecture

5 Why Run LLMs on Edge Devices?

6 Challenges and Bottlenecks of LLM Inference on Edge

7 Efficient LLM Inference Techniques for Edge

8 Speculative Decoding: The Basics

9 State-of-the-Art Speculative Decoding Techniques

10 Live Demo: Direct LLM Inference vs. Speculative Decoding

11 Can We Make the Drafter Better?

12 Improving the Drafter Through Knowledge Distillation

13 Key Takeaways

1.1 What Are Large Language Models (LLMs)?

Definition

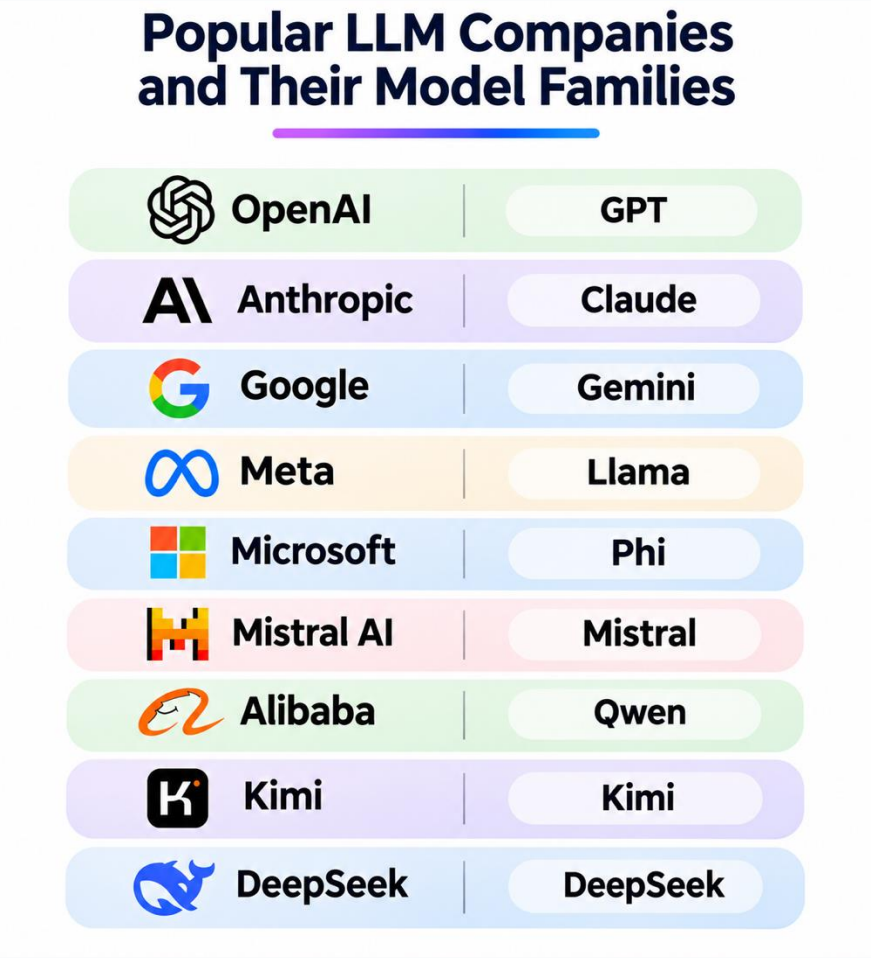
> A neural network trained on vast amounts of text to predict the next word. That skill lets it write, answer questions and follow instructions in plain language. [1]

Where you already use it

Chatbots
Answer questions, explain, summarize and translate.
e.g. ChatGPT, Claude, Gemini

Coding Assistants
Suggest, complete and fix code as you type.
e.g. Cursor, Claude Code, Codex

AI Agents
Plan steps and use tools such as search.
e.g. Manus, Devin, ChatGPT Agent



[1] Zhao et al., "A Survey of Large Language Models," 2023. arXiv:2303.18223

1.2 How Does an LLM Generate Text?

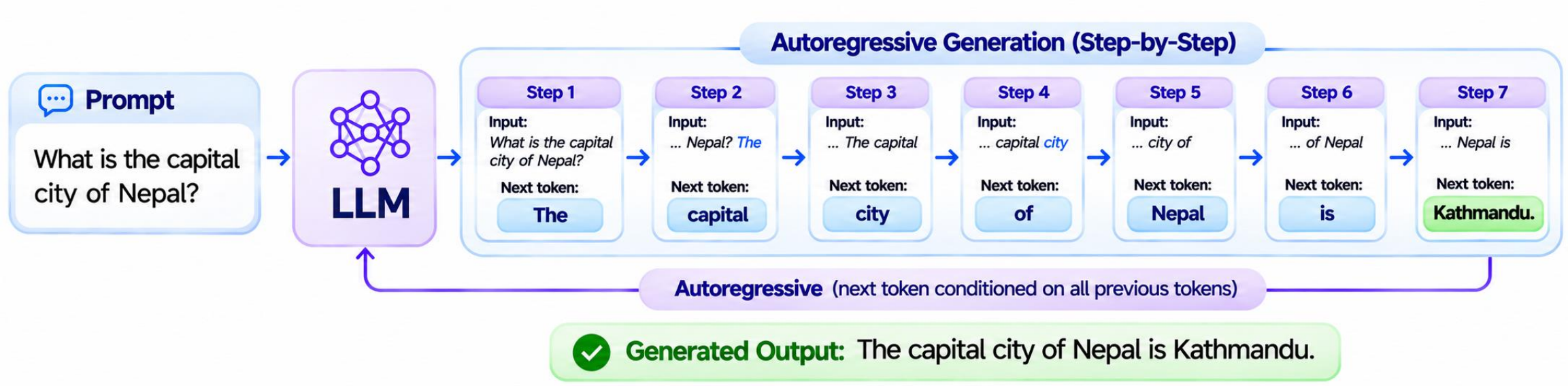
Autoregressive Generation: The model generates one token at a time. Each new token is predicted from everything written so far, then added to the text before the next one is predicted.

Sentence

What is the capital city of Nepal?

8 tokens (illustrative split)

What	is	the	capital	city	of	Nepal	?



Try the tokenizer (o200k_base): tiktokenizer.vercel.app/?model=o200k_base

1.3 Training vs. Inference

Training

Teaching the model

Once

Weeks on thousands of GPUs

Like: years of school before starting a job.

Inference

Using the model

Every Request

Fixed model, user waiting

Like: answering a question on the job.

> This talk is about inference: making each answer arrive faster.

2.1 Measuring Inference Performance

Time to first token (TTFT)

Wait until the first token appears.

$$\text{TTFT} = t_{first} - t_{request}$$

Time per output token (TPOT)

Average gap between tokens after the first.

$$\text{TPOT} = \frac{t_{last} - t_{first}}{N - 1}$$

End-to-end latency

Total time until the answer is complete.

$$\text{Latency} = \text{TTFT} + \text{TPOT} \times (N - 1)$$

Throughput

Tokens per second. For one user, total time = latency.

$$\text{Throughput} = \frac{\text{total output tokens}}{\text{total time}}$$

Example

Prompt	What is the capital city of Nepal?
Output	The capital city of Nepal is Kathmandu. (N = 8 tokens)
Given	**TTFT = 0.2 s, TPOT = 50 ms**
Question	**What are the end-to-end latency and the throughput?**
Answer	**Latency = 0.55 s · Throughput = 8 ÷ 0.55 ≈ 14.5 tokens/s**

2.2 Popular Inference Engines

Part 2

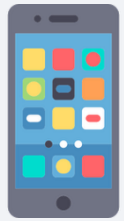
	Engine	Built for	Key idea
vLLM	**vLLM** [1]	GPU servers	Stores the KV cache in small blocks (PagedAttention), so less memory is wasted
SGL	**SGLang** [2]	GPU servers	Reuses the cached start of a prompt when requests share it (RadixAttention)
🤗	**Hugging Face TGI** [3]	GPU servers	Production server for Hugging Face models, with continuous batching
NVIDIA	**TensorRT-LLM** [4]	NVIDIA GPUs	Optimized NVIDIA GPU kernels, with quantization and in-flight batching
llama.cpp	**llama.cpp** [5]	CPUs, laptops, phones, edge boards	Lightweight C/C++ runtime for quantized (GGUF) models

All of them make each step cheaper. None changes the one-token-per-pass loop.

[1] vLLM: Kwon et al., SOSP 2023. github.com/vllm-project/vllm
[2] SGLang: Zheng et al., NeurIPS 2024. github.com/sgl-project/sglang
[3] Hugging Face TGI: github.com/huggingface/text-generation-inference
[4] TensorRT-LLM: github.com/NVIDIA/TensorRT-LLM
[5] llama.cpp: github.com/ggml-org/llama.cpp

3.1 What Counts as an Edge Device?

> **Edge Device:** a device that does its computing close to where data is created and used, at the edge of the network, instead of in a remote cloud data center. [1]



Phones

Always with the user. Strict battery and heat limits.

Smartphones, Tablets



Laptops

More memory and power. May have a GPU or NPU.

MacBooks, AI PCs



Edge boards

Small computers for labs, robots and kiosks.

NVIDIA Jetson, Raspberry Pi



Embedded

Devices that must decide on the spot.

Smart Cameras, Vehicles, Sensor Hubs

What they share: they run close to the user, with limited memory, compute and power.

[1] Shi et al., “Edge Computing: Vision and Challenges,” IEEE Internet of Things Journal, 2016. doi:10.1109/JIOT.2016.2579198

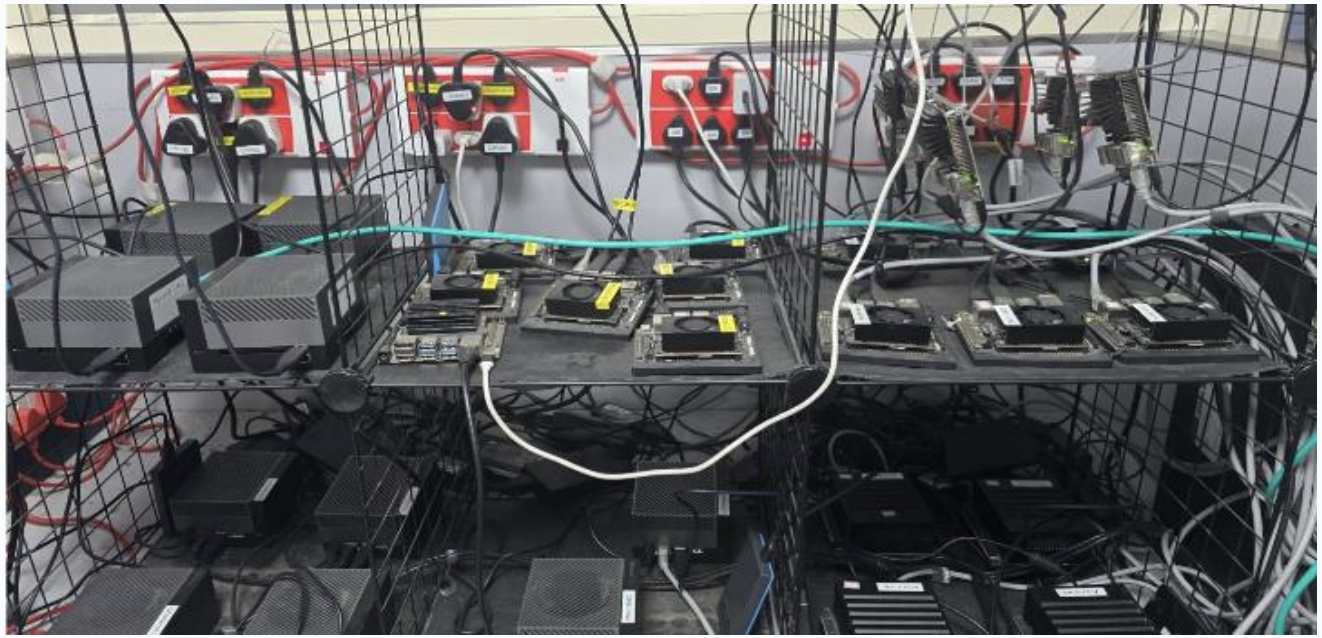
3.2 Our Lab's Edge Testbed



Raspberry Pi Cluster



Raspi-5 [1]



Jetson & Thor Cluster



AGX Xavier [2]



Orin Nano [3]



AGX Orin [3]



Thor [4]

[1] Raspberry Pi Ltd, Raspberry Pi 5 product page.
[2] NVIDIA, Jetson AGX Xavier series.
[3] NVIDIA, Jetson Orin series (Orin Nano, AGX Orin).
[4] NVIDIA, Jetson Thor.

3.3 What Is the Cloud?

> **Cloud:** computing power rented over the internet from large data centers, shared by many users. [1]

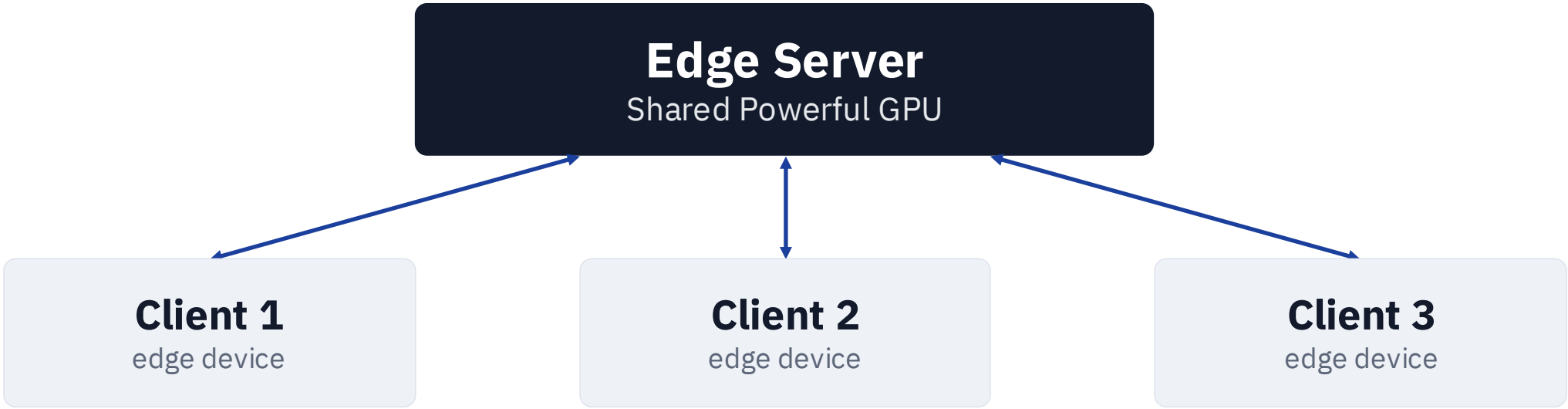


AWS Cloud Data Center [2]

[1] Mell and Grance, "The NIST Definition of Cloud Computing," NIST Special Publication 800-145, 2011.

[2] Bradley, "Amazon's Data Center Breakthrough Might Change The Future Of Cloud Storage," BGR, 2026.

4.1 One Shared Server, Many Edge Clients



Multi-tenant: many clients, one server, one model

What changes with many clients

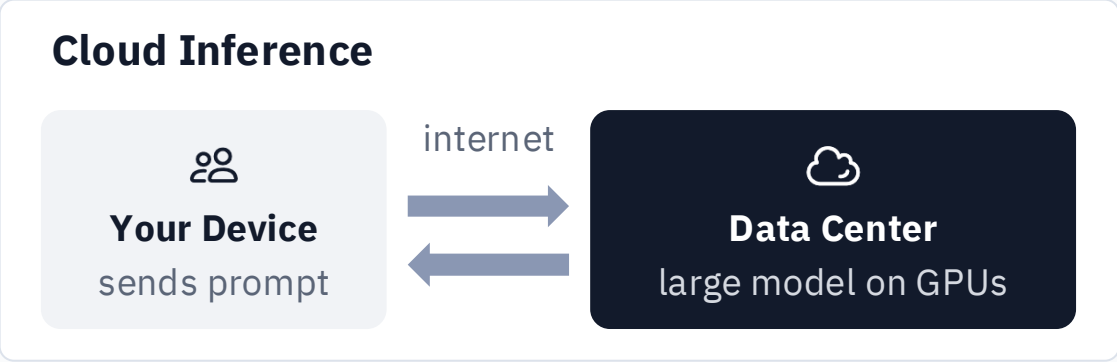
Shared GPU
Requests compete for the GPU

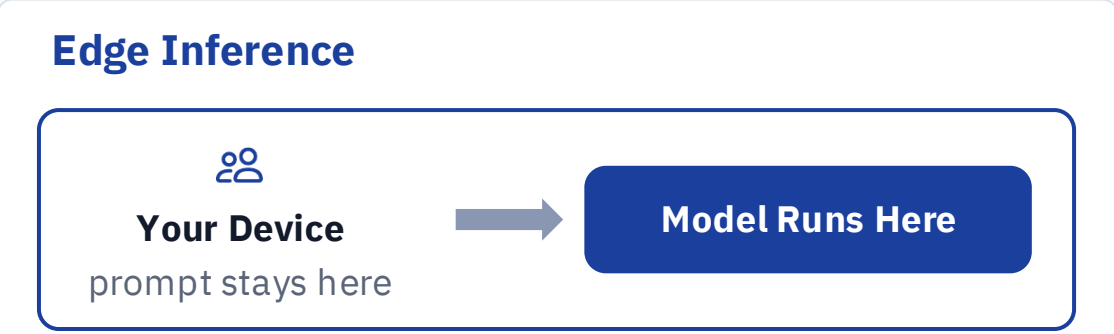
Queueing
Wait time grows with clients

Small clients
Too small to run the model

> **Our question:** how can small clients use a powerful shared server efficiently?

5.1 Cloud vs. Edge AI





	Cloud	Edge
Model Size	Very large	Small to medium
Delay	Network time plus queue	No network delay
Privacy	Data leaves the device	Data stays on the device
Works Offline	No	Yes
Cost	Pay per request	Uses hardware you own

5.2 Why Run AI at the Edge?



Privacy

Your data stays on your device.



Responsiveness

No round trip to a distant data center.



Works Offline

Runs without a network connection.



Lower Cost

No per-request cloud fees.

6.1 What Makes LLMs Expensive to Run?

Size

8B parameters take about 16 GB at 16-bit.

Llama 3.1 8B: 8B weights × 2 bytes = 16 GB.

Memory

Weights and KV cache must stay in fast memory.

Llama 3.1 8B: 128 KB of KV cache per token, 1 GB at 8K tokens.

Computation

About 2 operations per parameter, per token.

Llama 3.1 8B: About 16 GFLOPs per generated token.

Hardware

GPUs with fast memory and enough power.

Llama 3.1 8B: Fits a 24 GB RTX 4090, not an 8 GB phone.

7.1 Can We Make LLMs Faster at Edge?



Quantization

Cut weight precision (FP16 to INT8/INT4) to save memory and speed up inference.

e.g., AWQ, GPTQ



Knowledge Distillation

Transfer knowledge from a large teacher model to a smaller student model.

e.g., MiniLLM, Gemma 2



Pruning

Remove less important weights or structures to shrink the model and its compute.

e.g., SparseGPT, Wanda



Speculative Decoding

A small drafter proposes tokens. The larger target model verifies several in parallel.

e.g., EAGLE, Medusa

This list is not exhaustive. Others include KV caching, FlashAttention and continuous batching. [1]

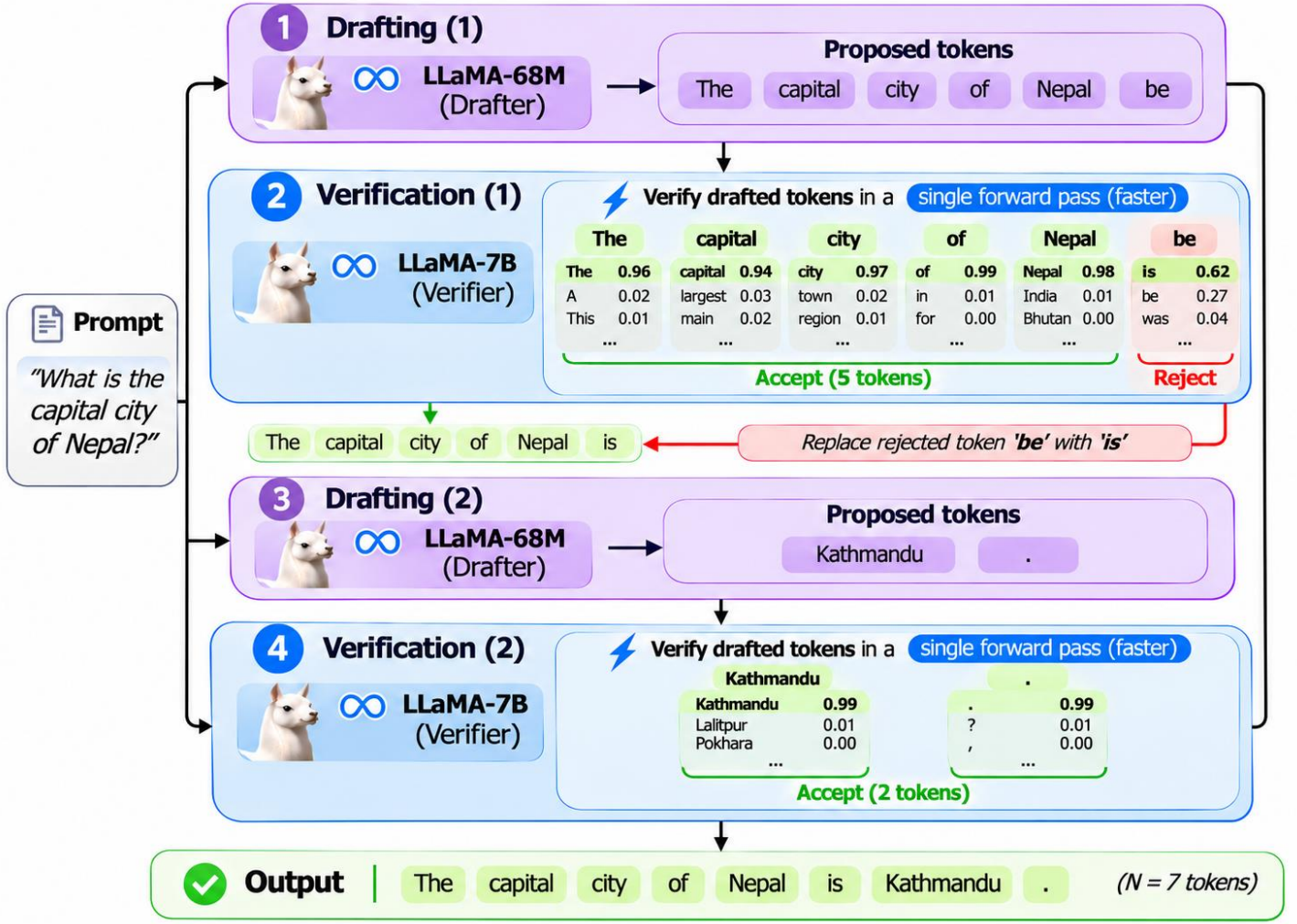
[1] Zhou et al., "A Survey on Efficient Inference for Large Language Models," arXiv:2404.14294, 2024.

8.1 Speculative Decoding: Draft, Then Verify

Speculative Decoding: a small model drafts several tokens, and a large model verifies them in one pass; this speeds up inference. [1]

Drafter: the small, fast model that guesses the next few tokens.

Verifier: the large target model that checks the guesses and has the final say.



[1] Leviathan et al., "Fast Inference from Transformers via Speculative Decoding," ICML 2023. arXiv:2211.17192

8.2 Standard Terms and Metrics

Term	Meaning	Symbol
Drafter [1]	The small, fast model that guesses tokens	M_q
Target Model	The large model whose output we want	M_p
Draft Length	Number of tokens guessed per round	γ

Acceptance rate (α) [1]

$$\alpha = \frac{\text{accept}}{\text{accept} + \text{reject}}$$

accept, reject: tokens the target model accepts or rejects.

Block Efficiency (τ) [1]

$$\tau(x) = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}$$

τ: tokens per iteration. **γ**: draft length. **α**: acceptance rate.

Speed-up [1]

$$\text{Speed} - \text{up} = \frac{\tau(x)}{\gamma c + 1}$$

c: cost ratio, one drafter run ÷ one target run.

[1] Hu et al., "AdaSPEC: Selective Knowledge Distillation for Efficient Speculative Decoders," NeurIPS 2025. arXiv:2510.19779

8.3 When Does Speculative Decoding Work Well?

	Fast Drafter	Slow Drafter
Often Agrees With the Target	**Big Win** Many tokens per pass, little overhead.	**Small Win** Good guesses, but drafting eats the gain.
Rarely Agrees With the Target	**Small Win** Cheap guesses, but most are thrown away.	**Can Be Slower** Than ordinary generation.

The best drafter is fast and agrees with the target model. [1, 2]

[1] Leviathan et al., "Fast Inference from Transformers via Speculative Decoding," ICML 2023. arXiv:2211.17192

[2] Xia et al., "Unlocking Efficiency in Large Language Model Inference: A Comprehensive Survey of Speculative Decoding," ACL Findings 2024. arXiv:2401.07851

8.4 Where Do the Two Models Run?

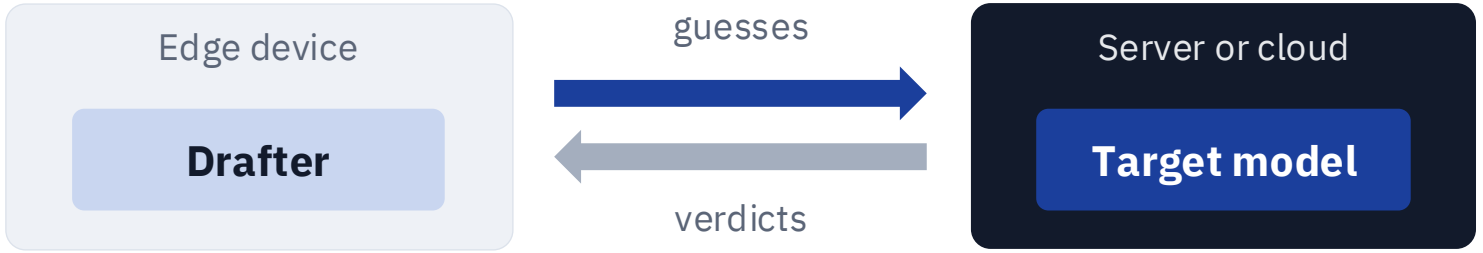
(a) Both on one device

Edge device (shared memory)

Drafter | **Target model**

+ Private, works offline – Both models must fit

(b) Draft on the device, verify on a server [1]



+ Much larger targets – Network round trip every round

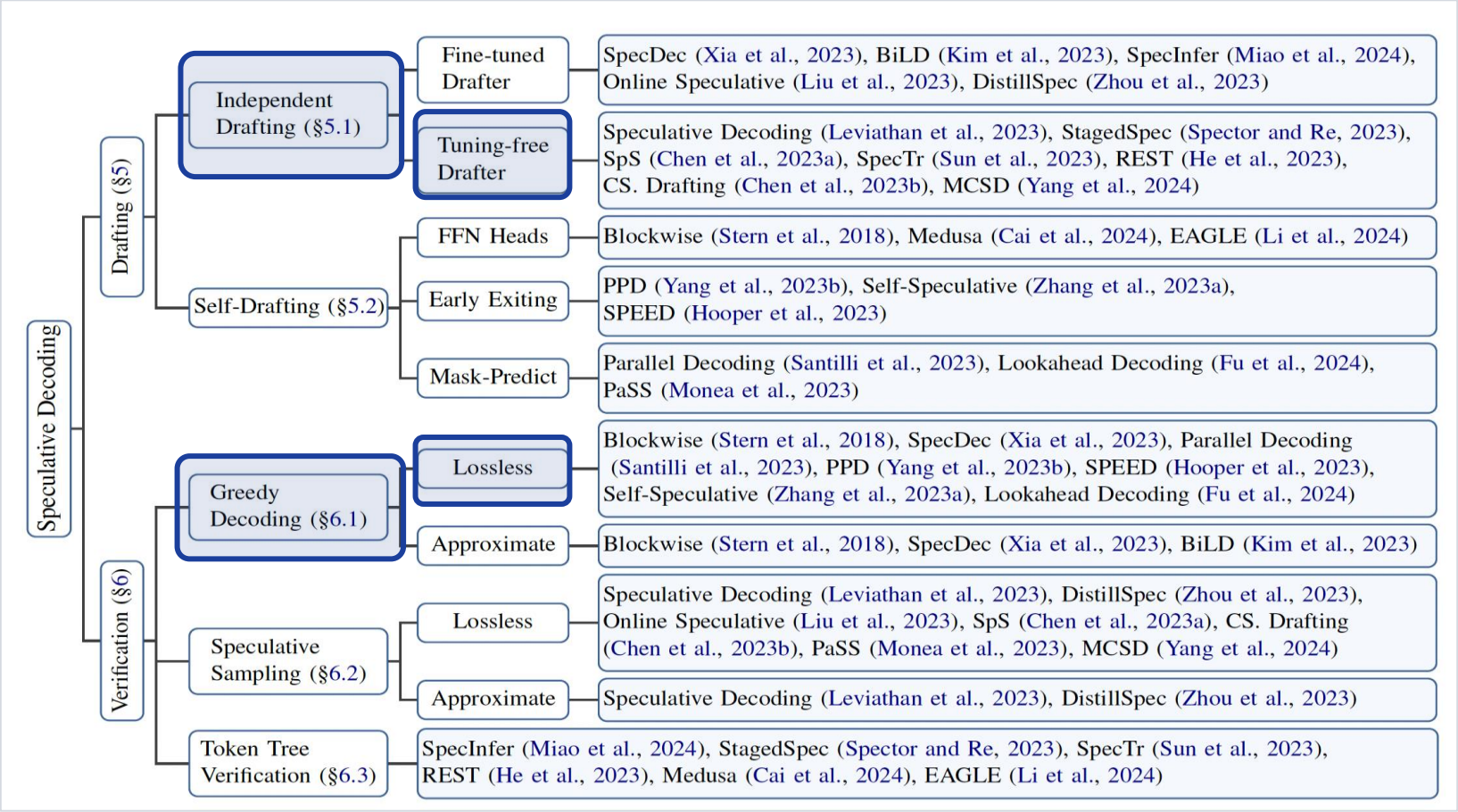
Fewer round trips pay off

In (b), each verification waits for the network, so accept more tokens per round.

The best option depends on memory, network and privacy needs.

[1] Venkatesha et al., “Fast and Cost-effective Speculative Edge-Cloud Decoding with Early Exits,” TMLR 2025. arXiv:2505.21594

9.1 Taxonomy of Speculative Decoding



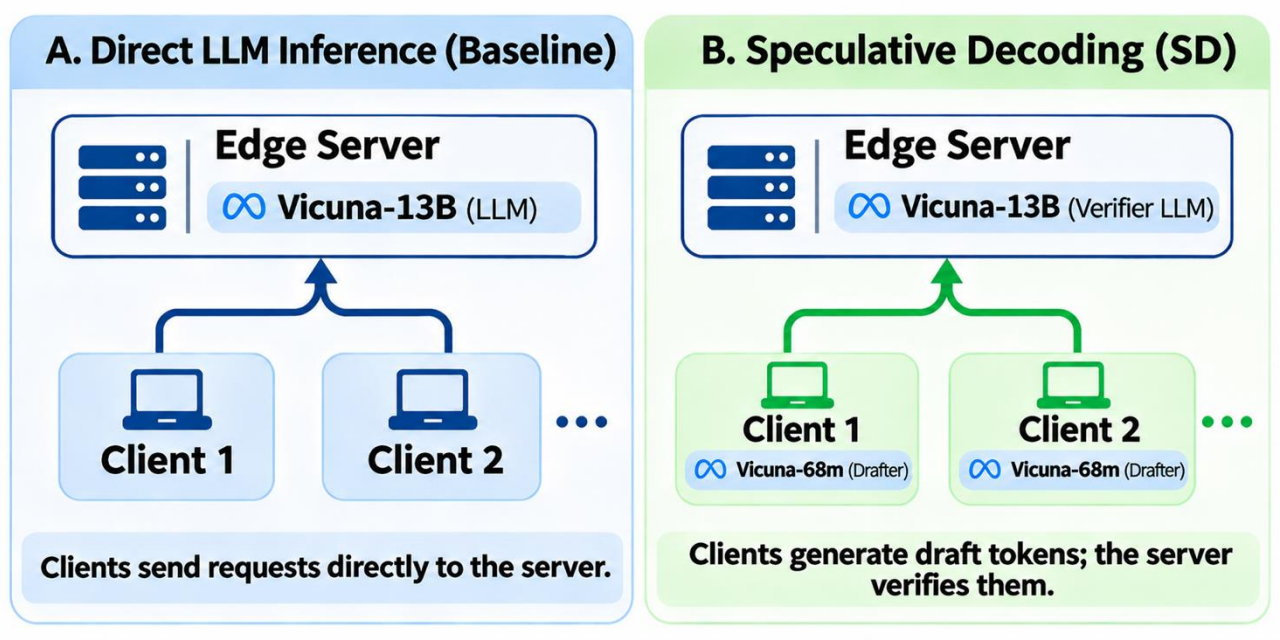
Taxonomy of Speculative Decoding [1]

Our focus

[1] Xia et al., "Unlocking Efficiency in Large Language Model Inference: A Comprehensive Survey of Speculative Decoding," ACL Findings 2024. arXiv:2401.07851

10.1 Live Demo: Direct Inference vs. Speculative Decoding

Central Question: How can multiple resource-constrained edge clients efficiently use a shared, powerful LLM server, and can speculative decoding make this interaction faster?



Draft model [3], target model [4]



1 x NVIDIA GeForce RTX 4090 [1]



3 x AGX Orin 64 [2]

[1] NVIDIA, GeForce RTX 4090 graphics card.
[2] NVIDIA, Jetson AGX Orin series.
[3] JackFram, Llama-68m model card.
[4] Meta, Llama-2-13b model card.

11.1 Choosing the Right Drafter

Drafter speed, agreement and overall speedup by drafter size
Relative index, illustrative shape, not measured data



Best drafter balances speed and agreement

Too Small
Fast, but guesses get rejected

Too Large
Agrees more, but costs too much time

> Drafter latency matters more than its standalone quality [1]
>
> The best choice depends on the hardware.

[1] Yan et al., "Decoding Speculative Decoding," NAACL 2025. arXiv:2402.01528

11.2 Can We Make the Drafter Better?

> **Can we make the drafter agree with the target more often, without making it slower?**

The problem with a bad drafter

It guesses poorly

Its tokens differ from the ones the target would pick.

The target rejects them

Verification keeps only the tokens before the first mismatch.

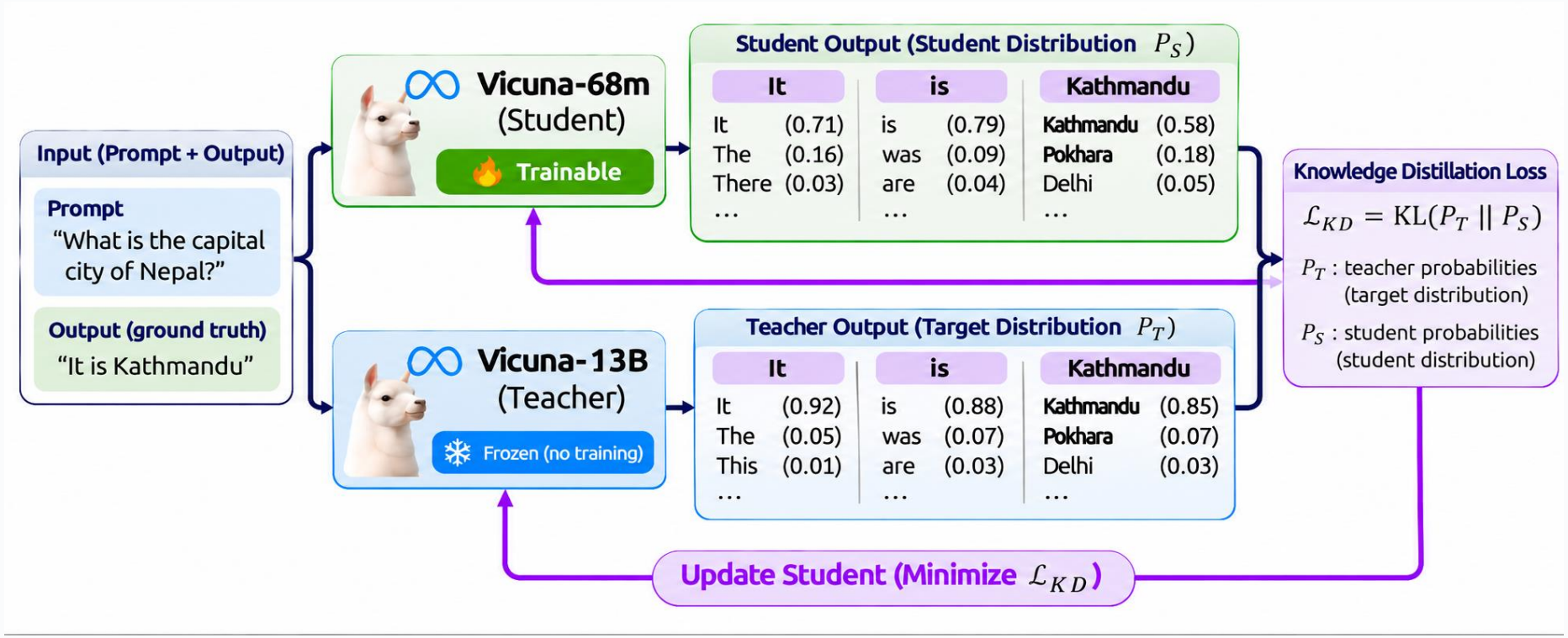
Multi-tenant: every rejection also costs a network round trip.

Little or no speedup

Each round still costs a drafter run and a target pass.

> **Next:** finetune the drafter to imitate the target, using knowledge distillation.

12.1 Knowledge Distillation: Learning From a Larger Model



Soft targets: the teacher gives a probability to every answer [1].

Training: the student learns to copy the teacher [1].

[1] Hinton et al., "Distilling the Knowledge in a Neural Network," NIPS Deep Learning Workshop 2014. arXiv:1503.02531

12.2 Distillation Methods and Loss Functions

Method	Divergence	Training data [6]
SeqKD [1]	FKL	Data generated by teacher M_p
Supervised KD [2]	FKL	Fixed dataset of input–output pairs
ImitKD [3]	FKL	Fixed dataset + data generated by M_q
f-Distill [4]	TVD	Data generated by M_q and M_p
On-policy GKD* [5]	FKL / JSD	On-policy data from student M_q

Forward KL (FKL) [6]

$$\mathrm{KL}(p \,\|\, q) = \Sigma_x\, p(x) \log [\, p(x) \,/\, q(x) \,]$$

Reverse KL (RKL) [6]

$$\mathrm{KL}(q \,\|\, p) = \Sigma_x\, q(x) \log [\, q(x) \,/\, p(x) \,]$$

Jensen–Shannon (JSD) [6]

$$\mathrm{JSD}(p \,\|\, q) = \tfrac{1}{2}\,\mathrm{KL}(p \,\|\, m) + \tfrac{1}{2}\,\mathrm{KL}(q \,\|\, m),\ \ m = \tfrac{1}{2}(p+q)$$

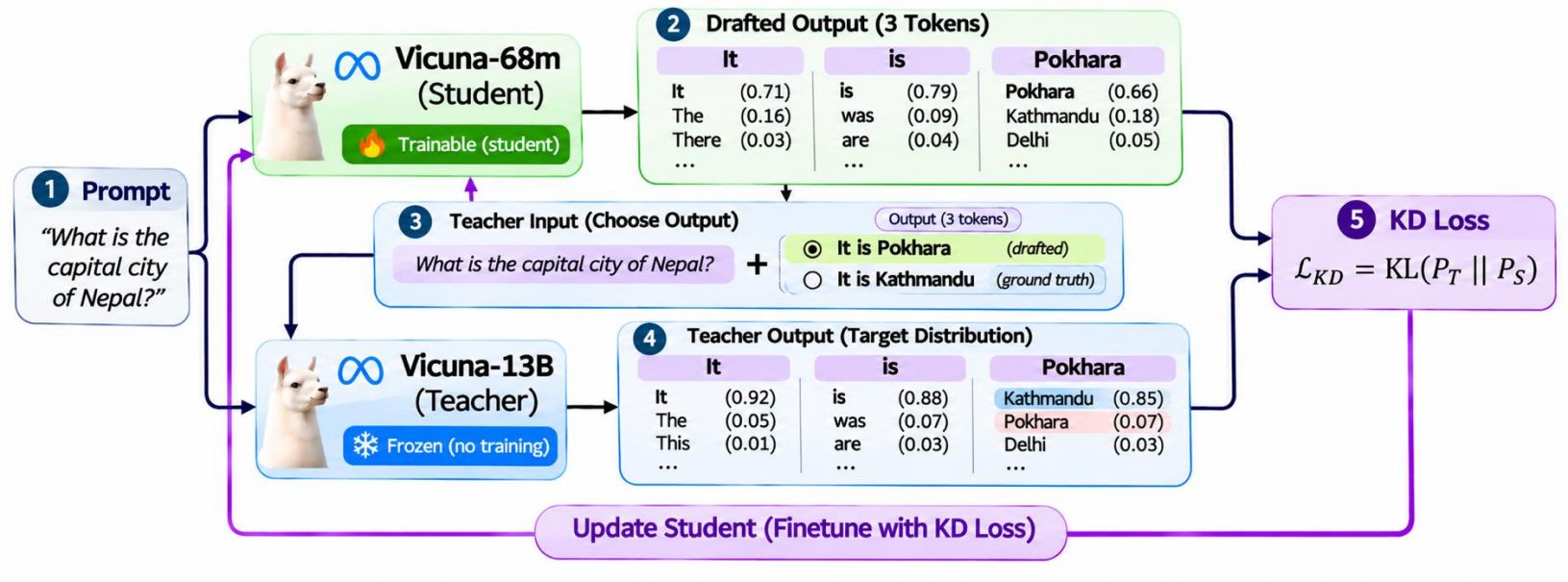
Total variation (TVD) [6]

$$\mathrm{TVD}(p \,\|\, q) = \tfrac{1}{2}\,\Sigma_x\, |\, p(x) - q(x)\,|$$

* DistillSpec found on-policy data, sampled from the drafter itself, gives the best distillation results [6].

[1] Kim and Rush, EMNLP 2016. arXiv:1606.07947
[2] Hinton et al., NIPS DL Workshop 2014. arXiv:1503.02531
[3] Lin et al., EMNLP 2020. arXiv:2009.07253
[4] Wen et al., ACL 2023. arXiv:2307.15190
[5] Agarwal et al., ICLR 2024. arXiv:2306.13649
[6] Zhou et al. (DistillSpec), ICLR 2024. arXiv:2310.08461

12.3 DistillSpec: Distilling the Drafter



1. **Prompt:** sample a prompt from the training set.
2. **Draft:** the drafter generates the next tokens itself, so training data matches what it will see at inference [1].
3. **Score:** the frozen teacher gives its probabilities for the same tokens.
4. **Compare:** at each position, measure how far the drafter's distribution is from the teacher's.
5. **Update:** minimise that divergence (KL, JSD or TVD, chosen per task) and change only the drafter [1].

[1] Zhou et al., "DistillSpec: Improving Speculative Decoding via Knowledge Distillation," ICLR 2024. arXiv:2310.08461

12.4 A Simple End-to-End Pipeline

1 **Collect prompts**

Gather prompts like the ones the device will really see.

2 **Generate Examples**

Let the target model answer them; its answers become training data. [2]

3 **Fine-tune the Drafter**

Train the small model to imitate those answers. [1]

4 **Deploy**

Pair the tuned drafter with the target on the device.

5 **Evaluate**

Measure speed and acceptance against the baselines.

Repeat when the target model, the device or the workload changes.

[1] Zhou et al., “DistillSpec: Improving Speculative Decoding via Knowledge Distillation,” ICLR 2024. arXiv:2310.08461

[2] Kim and Rush, “Sequence-Level Knowledge Distillation,” EMNLP 2016. arXiv:1606.07947

12.5 How Do We Know It Works?

A **Ordinary Generation**

Target model alone (baseline).

B **Standard Drafter**

Speculative decoding with an off-the-shelf drafter.

C **Fine-tuned Drafter**

Speculative decoding with a drafter tuned to the target.

Keep fixed: prompts, target model, device, decoding settings.

Check: B and C must output the same text as A (greedy decoding).

10–45%

faster than standard speculative decoding [1]

Reported for distilled drafters.

Caveat: not for a multi-tenant system.

[1] Zhou et al., "DistillSpec: Improving Speculative Decoding via Knowledge Distillation," ICLR 2024. arXiv:2310.08461

13.1 Key Takeaways

1

Edge generation is slow

Each token re-reads all of the model's weights from memory, so speed is set by memory, not compute.

2

A small drafter speeds it up

The drafter guesses several tokens ahead. The target model checks them all in one pass.

3

Multi-tenant needs a better drafter

Each rejection costs a network round trip, so a drafter that agrees with the target matters most there.

4

Distillation improves the drafter

Fewer rejections help on the edge and in multi-tenant serving, where each one costs a round trip.

Thank you.
Questions?

Manjil Nepal

Research Associate, Indian Institute of Science, Bangalore

manjil.nepal03@gmail.com
manjilnepal.github.io

AI Disclosure: These slides were prepared with the assistance of Claude (Anthropic) and GPT (OpenAI). All content has been reviewed and verified by the author.